Scheduling with Two Search Models

OR-tree Feasibility ⟶ Set-based Optimization

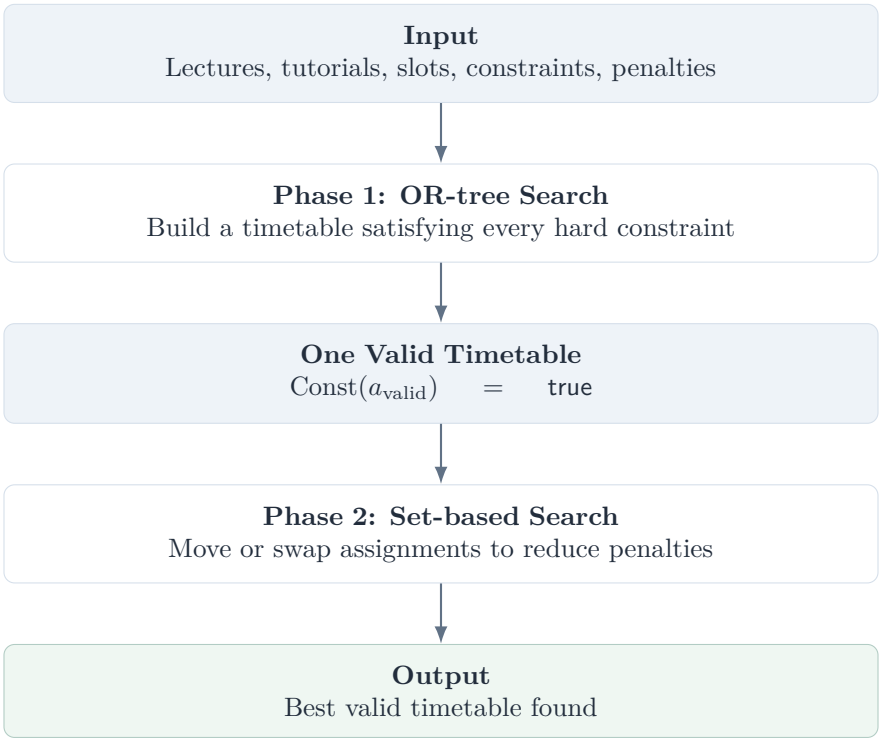
CPSC 433 | Team Explanation and Algorithms

The entire idea in two sentences.
First, we build *one* valid timetable, assigning classes one at a time in an OR-tree.
Then, starting from that complete timetable, we use set-based moves and swaps to lower the soft-constraint penalty without breaking any hard constraints.

1. Overview of Our Approach

Our solution separates the problem into two tasks:

1. **Constraint Satisfaction:** Find one valid timetable using OR-tree search.
2. **Optimization:** Improve that timetable using set-based search.



Important distinction.
The OR-tree does not need to generate all valid timetables. It stops when it finds one complete valid timetable.
The set-based search then improves that timetable. It does not construct a new one from scratch.

2. Common Definitions

Before defining either model, we establish the scheduling problem.

Symbol	Meaning
L	Set of all lecture sections.
T	Set of all tutorials and labs.
$X = L \cup T$	All components requiring scheduling.
S	Available weekly time slots.
$x \in X$	An individual lecture or tutorial.
$s \in S$	An individual time slot.
$D(x)$	Allowed time slots for component x.
a	Assignment of components to slots.
U	Set of unscheduled components.
$\text{dom}(a)$	Components already scheduled.
$\text{Consistent}(a)$	Checks whether a partial assignment violates any hard constraint.
$\text{Const}(a)$	Checks whether a complete timetable satisfies all hard constraints.
$\text{Eval}(a)$	Calculates the soft-constraint penalty.

A complete timetable is a function:

$$\boxed{a : X \to S} \tag{1}$$

Every component must be assigned to one of its allowed time slots:

$$\forall x \in X, \quad a(x) \in D(x) \tag{2}$$

Our final optimization objective is:

$$\boxed{\min_{a:X\to S} \text{Eval}(a)} \tag{3}$$

Subject to:

$$\boxed{\text{Const}(a) = \mathsf{true}} \tag{4}$$

In simple terms, we want a valid timetable with the smallest possible penalty.

3. Worked Scheduling Example

We will use four components from CPSC 231.

Symbol	Component	Allowed Slots
L_1	CPSC 231 LEC 01	MWF 9–10
L_2	CPSC 231 LEC 02	MWF 9–10, 10–11, 11–12
T_1	CPSC 231 TUT 01	TR 10–11
T_2	CPSC 231 TUT 02	MW 11–12

For this simplified example, assume:

- A time slot supports at most two lectures.
- A tutorial slot supports at most one tutorial.
- Both tutorials serve both lecture sections.
- Tutorials cannot overlap either lecture section.
- Scheduling both lecture sections at the same time creates a soft penalty of 5.

These are assumptions for the worked example, not the complete project constraints.

Our simplified evaluation function is:

$$\boxed{\text{Eval}(a) = 5 \cdot \mathbf{1}[a(L_1) = a(L_2)]} \tag{5}$$

Where:

$$\mathbf{1}[P] = \begin{cases} 1 & \text{if proposition } P \text{ is true} \\ 0 & \text{otherwise} \end{cases} \tag{6}$$

4. Phase 1: OR-tree Search

The OR-tree's job is to find a complete timetable satisfying all hard constraints.

4.1 Search Model

Following Lecture 11:

$$\boxed{A_\vee = (S_\vee, T_\vee)} \tag{7}$$

Where:

- $A_\vee$ is the OR-tree search model.
- $S_\vee$ is the set of OR-tree states.
- $T_\vee$ is the transition relation.

4.2 Problem Descriptions

We define an individual scheduling problem:

$$\boxed{pr = (a, U)} \tag{8}$$

Where:

- pr is a problem description.
- a is the partial timetable.
- U contains unscheduled components.

The set of possible problem descriptions is:

$$\boxed{Prob = \{(a, U) \mid a \text{ is a partial assignment}, \quad U = X \setminus \text{dom}(a)\}} \tag{9}$$

Two conditions must always hold:

$$\boxed{\text{dom}(a) \cap U = \varnothing} \tag{10}$$

$$\boxed{\text{dom}(a) \cup U = X} \tag{11}$$

These ensure every component is either scheduled or unscheduled, but never both.

4.3 OR-tree Nodes

According to Lecture 11, an OR-tree node is:

$$\boxed{(pr, sol)} \tag{12}$$

For our scheduling problem:

$$\boxed{((a, U), sol)} \tag{13}$$

Where:

$$sol \in \{\mathsf{yes}, ?, \mathsf{no}\} \tag{14}$$

The three values mean:

- yes: A solution has been found.
- ?: The problem is not yet resolved.
- no: The problem cannot be completed.

Importantly, the search state is the **entire OR-tree**, not one individual node.

$$\boxed{S_{\vee} \subseteq Otree} \tag{15}$$

Here *Otree* represents the possible OR-tree structures described in Lecture 11.

4.4 Alternatives Relation

The alternatives relation, *Altern*, generates different ways to continue scheduling.

Given:

$$pr = (a, U) \tag{16}$$

Choose an unscheduled component:

$$x \in U \tag{17}$$

Define its currently legal slots:

$$\boxed{D_a(x) = \{s \in D(x) \mid \text{Consistent}(a \cup \{x \mapsto s\})\}} \tag{18}$$

Where:

- $D(x)$ is the original set of allowed slots.
- $D_a(x)$ contains slots that remain legal under the existing partial timetable.
- Consistent checks hard constraints involving assignments made so far.

For every candidate slot $s_i \in D_a(x)$, create a new problem:

$$\boxed{pr_i = (a \cup \{x \mapsto s_i\}, U \setminus \{x\})} \tag{19}$$

If there are k legal slots:

$$\boxed{\mathit{Altern}(pr, pr_1, \ldots, pr_k)} \tag{20}$$

Here $k = |D_a(x)|$.

Each alternative assigns one more component.

4.5 How the Assignment Grows

At each successful assignment:

$$\boxed{a_{i+1} = a_i \cup \{x_i \mapsto s_i\}} \tag{21}$$

$$\boxed{U_{i+1} = U_i \setminus \{x_i\}} \tag{22}$$

Where:

- i is the current step.
- x_i is the selected component.
- s_i is the selected time slot.

Therefore:

$$|\operatorname{dom}(a_{i+1})| = |\operatorname{dom}(a_i)| + 1 \tag{23}$$

$$|U_{i+1}| = |U_i| - 1 \tag{24}$$

4.6 Solved and Unsolvable Problems

A scheduling problem is solved when:

$$\boxed{Solved(pr) \iff U = \varnothing \wedge \text{Const}(a) = \mathsf{true}} \tag{25}$$

One sufficient condition for an unsolvable problem is:

$$\boxed{\exists x \in U : D_a(x) = \varnothing} \tag{26}$$

This means an unscheduled component has no legal time slot remaining.

A problem can also be unsolvable when all of its alternatives have been exhausted.

4.7 OR-tree Transitions

Lecture 11 defines transitions using the extension relation $Erw_\vee$.

$$\boxed{T_\vee = \{(q, q') \mid Erw_\vee(q, q') \text{ or } Erw^*_\vee(q, q')\}} \tag{27}$$

Where q and q' represent entire tree states.

The main extension cases are:

Rule 1: Solved Leaf

$$(pr, ?) \longrightarrow (pr, \mathsf{yes}) \tag{28}$$

Rule 2: Unsolvable Leaf

$$(pr, ?) \longrightarrow (pr, \mathsf{no}) \tag{29}$$

Rule 3: Expand a Leaf

$$\boxed{(pr, ?) \longrightarrow (pr, ?, (pr_1, ?), \ldots, (pr_k, ?))} \tag{30}$$

The extension rules may also be applied to leaves deeper in the search tree.

Why we do not need backtracking.
If a branch fails, we mark that node no and select another unfinished leaf from the OR-tree.
We do not need an explicit operation that undoes an earlier tree transition.

5. OR-tree Search Process

Following the lecture:

$$\boxed{P_\vee = (A_\vee, Env_\vee, K_\vee)} \tag{31}$$

Where:

- $P_\vee$ is the search process.
- $A_\vee$ is the search model.
- $Env_\vee$ stores scheduling information.
- $K_\vee$ is the search control.

5.1 Our Search Control

We propose two main techniques.

1. Depth-first leaf selection

Explore one branch deeply before selecting a different branch.

2. Minimum Remaining Values (MRV)

Choose the unscheduled component with the fewest currently legal slots:

$$\boxed{x^* = \arg\min_{x \in U} |D_a(x)|} \tag{32}$$

Where x^* is the chosen component.

For this initial design, we order slot choices by time and explore the earliest legal slot first.

We are deliberately not using forward checking in this version.

5.2 Search Instance

Following Lecture 11:

$$\boxed{Inst_\vee = (q_0, G_\vee)} \tag{33}$$

Where:

- q_0 is the initial OR-tree state.
- $G_\vee$ is the goal condition.

Initially:

$$\boxed{a_0 = \varnothing} \tag{34}$$

$$\boxed{U_0 = X} \tag{35}$$

$$\boxed{pr_0 = (a_0, U_0)} \tag{36}$$

$$\boxed{q_0 = (pr_0, ?)} \tag{37}$$

The search terminates when a solved branch is found, or all alternatives have been exhausted.

The outcome is:

$$\boxed{Outcome_{\vee}(q) = \begin{cases} a_{\text{valid}} & \text{if a solved branch exists} \\ \textsf{FAILURE} & \text{if every alternative fails} \end{cases}} \tag{38}$$

6. Algorithm 1: OR-tree Feasibility Search

Require: Components X, domains D, hard constraints Consistent
Ensure: One valid timetable or FAILURE

$q \leftarrow ((\varnothing, X), ?)$
while the tree q contains an unresolved leaf **do**
  $v \leftarrow$ deepest unresolved leaf of q
  $(a, U) \leftarrow$ problem description stored at v
  if $U = \varnothing$ **then**
    if Const$(a) =$ true **then**
      Mark v as yes
      return a
    else
      Mark v as no
      continue
    end if
  end if
  $x \leftarrow \arg\min_{y \in U} |D_a(y)|$
  $C \leftarrow \{s \in D(x) : \text{Consistent}(a \cup \{x \mapsto s\})\}$
  if $C = \varnothing$ **then**
    Mark v as no
  else
    Order C by slot time
    for all $s \in C$ **do**
      $a' \leftarrow a \cup \{x \mapsto s\}$
      $U' \leftarrow U \setminus \{x\}$
      Add child $((a', U'), ?)$ to v
    end for
  end if
end while
return FAILURE

The algorithm operates on the OR-tree. When one alternative fails, another unfinished leaf can be selected.

No separate backtracking operation is needed.

7. Worked OR-tree Derivation

For readability, define:

$$M9 = \text{MWF 9–10}$$
$$M10 = \text{MWF 10–11}$$
$$M11 = \text{MWF 11–12}$$
$$R10 = \text{TR 10–11}$$
$$W11 = \text{MW 11–12}$$

7.1 Initial Problem

$$a_0 = \varnothing \tag{39}$$

$$U_0 = \{L_1, L_2, T_1, T_2\} \tag{40}$$

7.2 Step 1

Assign LEC 01 to MWF 9–10.

$$a_1 = \{L_1 \mapsto M9\} \tag{41}$$

$$U_1 = \{L_2, T_1, T_2\} \tag{42}$$

7.3 Step 2

Assign TUT 01 to TR 10–11.

$$a_2 = a_1 \cup \{T_1 \mapsto R10\} \tag{43}$$

$$U_2 = \{L_2, T_2\} \tag{44}$$

7.4 Step 3

Assign TUT 02 to MW 11–12.

$$a_3 = a_2 \cup \{T_2 \mapsto W11\} \tag{45}$$

$$U_3 = \{L_2\} \tag{46}$$

7.5 Step 4

Only LEC 02 remains unscheduled.

Its original domain is:

$$D(L_2) = \{M9, M10, M11\} \tag{47}$$

However:

$$\boxed{D_{a_3}(L_2) = \{M9, M10\}} \tag{48}$$

MWF 11–12 is excluded because it overlaps TUT 02 on Monday and Wednesday.

Suppose the OR-tree tries MWF 9–10 first.

$$a_4 = a_3 \cup \{L_2 \mapsto M9\} \tag{49}$$

$$\boxed{U_4 = \varnothing} \tag{50}$$

Every component has now been assigned.

$$\boxed{\text{Const}(a_4) = \textsf{true}} \tag{51}$$

The leaf is marked:

$$(pr_4, ?) \to (pr_4, \textsf{yes}) \tag{52}$$

End of Phase 1
The OR-tree returns one valid timetable:

LEC 01	MWF 9–10
LEC 02	MWF 9–10
TUT 01	TR 10–11
TUT 02	MW 11–12

Its penalty is:

$$\boxed{\text{Eval}(a_4) = 5}$$

The two lecture sections share the same time.
This is allowed by our assumed hard constraints, but creates a soft-constraint penalty.

8. Phase 2: Set-based Search

Now we already have a valid timetable.

Our goal is to improve its soft-constraint penalty while keeping it valid.

8.1 Search Model

Following Lecture 06:

$$\boxed{A_{set} = (S_{set}, T_{set})} \tag{53}$$

Where:

- A_{set} is the set-based search model.
- S_{set} is the set of possible states.
- T_{set} is the transition relation.

8.2 Facts

Set-based search represents knowledge as a set of facts.

We define:

$$\boxed{F = \{\text{Assigned}(x, s) \mid x \in X,\ s \in D(x)\}} \tag{54}$$

Here:

$$\text{Assigned}(x, s) \tag{55}$$

means component x has been assigned to time slot s.

For example:

$$\text{Assigned}(L_1, M9) \tag{56}$$

means CPSC 231 LEC 01 is assigned to MWF 9–10.

8.3 States

The lecture defines:

$$S_{set} \subseteq 2^F \tag{57}$$

We define our particular state space as:

$$\boxed{S_{set} = \left\{ q \subseteq F \;\middle|\; \begin{array}{l} q \text{ assigns every } x \in X \text{ exactly once,} \\ \text{Const}(a_q) = \mathsf{true} \end{array} \right\}} \tag{58}$$

Where:

- q is a set of assignment facts.
- a_q is the timetable represented by q.

Every set-based state represents a complete, valid timetable.

8.4 Extension Rules

Lecture 06 defines extension rules:

$$\boxed{A \to B} \tag{59}$$

Where:

- A contains facts to remove.
- B contains replacement facts.

We propose two extension rules.

Rule 1: Move

$$\boxed{\{\text{Assigned}(x, s_1)\} \to \{\text{Assigned}(x, s_2)\}} \tag{60}$$

Move component x from s_1 to s_2.

Rule 2: Swap

$$\boxed{\begin{array}{l}\{\text{Assigned}(x, s_1), \text{Assigned}(y, s_2)\} \\ \quad \longrightarrow \{\text{Assigned}(x, s_2), \text{Assigned}(y, s_1)\}\end{array}} \tag{61}$$

Swap the time slots assigned to components x and y.

Both rules must preserve every hard constraint. Preassigned components cannot be moved.

8.5 Transitions

Following Lecture 06:

$$\boxed{q' = (q \setminus A) \cup B} \tag{62}$$

Where:

- q is the current state.
- A is the set of removed facts.
- B is the set of inserted facts.
- q' is the new state.

The transition relation is:

$$\boxed{\begin{aligned} T_{set} = \{(q, q') \mid \exists (A \to B) \in EXT, \\ A \subseteq q, \\ q' = (q \setminus A) \cup B, \\ q' \in S_{set}\} \end{aligned}} \tag{63}$$

The condition $q' \in S_{set}$ ensures we never move into an invalid timetable.

9. Set-based Search Process

Following Lecture 06:

$$\boxed{P_{set} = (A_{set}, Env_{set}, K_{set})} \tag{64}$$

Where:

- P_{set} is the search process.
- A_{set} is the search model.
- Env_{set} stores relevant information.
- K_{set} selects the next transition.

9.1 Weight Function

The professor introduces f_{weight} to evaluate possible extension rules.

We define:

$$\boxed{f_{weight}(A, B, e) = \text{Eval}\left(a_{(q \setminus A) \cup B}\right)} \tag{65}$$

Where the environment e includes the current state q and data required to evaluate the resulting timetable.

The weight is the soft-constraint penalty after performing the move.

Lower weight means a better timetable.

9.2 Tie-breaking Function

If multiple extension rules produce the same minimum weight, use:

$$\boxed{f_{select}} \tag{66}$$

Our initial design selects randomly among the equally best extensions.

9.3 Hill-climbing Control

For simplicity, our first implementation uses strict hill climbing.

Define the neighborhood:

$$\boxed{N(q) = \{q' \in S_{set} \mid (q, q') \in T_{set},\ q' \neq q\}} \tag{67}$$

Where $N(q)$ contains all valid timetables reachable by one move or swap.

Choose the neighbor with the lowest penalty.

We only perform a move if:

$$\boxed{\min_{q' \in N(q)} \text{Eval}(a_{q'}) < \text{Eval}(a_q)} \tag{68}$$

If there is no improving neighbor, the algorithm stops.

This is called a **local optimum**.

It may not be the global optimum.

9.4 Search Instance

Following Lecture 06:

$$\boxed{Inst_{set} = (q_0, G_{set})} \tag{69}$$

Our initial state is generated from the OR-tree solution:

$$\boxed{q_0 = \{\text{Assigned}(x, a_{\text{valid}}(x)) \mid x \in X\}} \tag{70}$$

The search terminates when:

- The penalty reaches zero, or
- No improving neighbor remains.

A practical implementation may also use a time or iteration limit.

10. Algorithm 2: Set-based Optimization

Require: Valid timetable a_{valid}, allowed slots, hard constraints, Eval
Ensure: Improved valid timetable
1: $q \leftarrow \{\text{Assigned}(x, a_{\text{valid}}(x)) : x \in X\}$
2: **while** $\text{Eval}(a_q) > 0$ **do**
3: $N \leftarrow \{q' \in S_{set} : (q, q') \in T_{set},\ q' \neq q\}$
4: **if** $N = \varnothing$ **then**
5: **return** a_q
6: **end if**
7: $m \leftarrow \min\{\text{Eval}(a_{q'}) : q' \in N\}$
8: **if** $m \geq \text{Eval}(a_q)$ **then**

```
        return a_q
    end if
    B ← {q' ∈ N : Eval(a_q') = m}
    q ← f_select(B)
end while
return a_q
```

For readability, the algorithm above uses f_{select} to choose a neighboring state.

In Lecture 06, f_{select} formally chooses among equally weighted extension rules that produce those states.

11. Worked Set-based Derivation

Our OR-tree returned:

$$\begin{aligned} q_0 = \{ &\text{Assigned}(L_1, M9), \\ &\text{Assigned}(L_2, M9), \\ &\text{Assigned}(T_1, R10), \\ &\text{Assigned}(T_2, W11)\} \end{aligned} \tag{71}$$

Its initial penalty is:

$$\boxed{\text{Eval}(a_{q_0}) = 5} \tag{72}$$

The optimizer considers moving LEC 02 from MWF 9–10 to MWF 10–11.

Define:

$$A = \{\text{Assigned}(L_2, M9)\} \tag{73}$$

$$B = \{\text{Assigned}(L_2, M10)\} \tag{74}$$

Apply the set-based transition:

$$\boxed{q_1 = (q_0 \setminus A) \cup B} \tag{75}$$

The new state becomes:

$$\begin{aligned} q_1 = \{ &\text{Assigned}(L_1, M9), \\ &\text{Assigned}(L_2, M10), \\ &\text{Assigned}(T_1, R10), \\ &\text{Assigned}(T_2, W11)\} \end{aligned} \tag{76}$$

This timetable satisfies all hard constraints.

The two lecture sections now occupy different time slots.

Therefore:

$$\boxed{\text{Eval}(a_{q_1}) = 0} \tag{77}$$

Final Improvement

$$\boxed{5 \longrightarrow 0}$$

The OR-tree found a valid solution with penalty 5.
The set-based search changed one assignment and reduced the penalty to 0.

11.1 Final Timetable

Component	OR-tree Output	Final Timetable
CPSC 231 LEC 01	MWF 9–10	MWF 9–10
CPSC 231 LEC 02	MWF 9–10	**MWF 10–11**
CPSC 231 TUT 01	TR 10–11	TR 10–11
CPSC 231 TUT 02	MW 11–12	MW 11–12
Soft penalty	5	**0**

12. Strengths and Limitations

12.1 Strengths

- The two models have clearly separated responsibilities.
- The OR-tree focuses on satisfying hard constraints.
- Set-based search focuses on minimizing soft penalties.
- The design closely follows the search-model terminology from Lectures 06 and 11.
- The set-based optimizer always maintains a valid timetable.

12.2 Limitations

- OR-tree search can require exploring many alternatives.
- Storing the OR-tree can consume substantial memory.
- Hill climbing may get stuck at a local optimum.
- Single moves and swaps may be insufficient for some schedules.
- The final timetable is not guaranteed to have the globally minimum possible penalty.

Possible future improvements include:

- Forward checking for OR-tree search.
- Least Constraining Value slot ordering.

- Multi-component assignment moves.
- Random restarts.
- Simulated annealing.

13. Three Important Invariants

These are conditions that our implementation should maintain.

Invariant 1: OR-tree Partition

$$\boxed{\mathrm{dom}(a) \cap U = \varnothing} \tag{78}$$

$$\boxed{\mathrm{dom}(a) \cup U = X} \tag{79}$$

Every component is either already assigned or waiting to be assigned.

Invariant 2: OR-tree Legality

Every generated child must satisfy:

$$\boxed{\mathrm{Consistent}(a) = \mathsf{true}} \tag{80}$$

Each child adds exactly one assignment.

Invariant 3: Set-based Validity

Every state $q \in S_{set}$ represents a complete valid timetable:

$$\boxed{\mathrm{Const}(a_q) = \mathsf{true}} \tag{81}$$

All moves and swaps must preserve this property.

Final Summary

OR-tree Search
Start with:

$$a = \varnothing, \qquad U = X$$

Choose an unscheduled component, generate alternative legal assignments, and grow the search tree. Stop when:

$$U = \varnothing \quad \wedge \quad \mathrm{Const}(a) = \mathsf{true}$$

Return one valid timetable.

Set-based Search
Convert the valid timetable into a set of assignment facts:

$$q = \{\text{Assigned}(x, s), \ldots\}$$

Generate valid moves and swaps, evaluate their penalties, and continue improving while possible.

The key distinction
A valid timetable is not necessarily an optimal timetable.
The OR-tree finds something that works.
The set-based search tries to make it better.